Ex. No. 3	Add Interactivity to App – Input and Forms
Date of Exercise	31 October 2026
GitHub Code Link (Keep as public)	https://github.com/jeropradeepsinghr14/24CS2016-Building-Applications-using-Flutter

Aim

To develop an interactive Flutter application that displays a predefined list of products and allows users to search and filter products using input widgets, thereby providing an interactive user experience.

Description

Input widgets allow users to provide information or make selections in an application. Flutter provides widgets such as TextField, ChoiceChip and DropdownButtonFormField to collect user input. Event callbacks respond to user actions, while setState() updates the user interface when the application state changes.

This experiment implements an interactive e-commerce product catalog named "ShopEase Interactive Store". A predefined list of products is displayed as cards in a grid. Users can enter a product name in the search field, select a category, choose a sorting option and clear the applied filters.

The main widgets and concepts used are:

- TextField: Accepts search text from the user. Its onChanged callback updates the search query as the user types.
- ChoiceChip: Allows the user to select a product category such as All, Electronics, Fashion or Accessories.
- DropdownButtonFormField: Provides sorting choices including default order, price from low to high, price from high to low and rating from high to low.
- TextEditingController: Controls the search field and allows its text to be cleared programmatically.
- setState(): Updates the application state and rebuilds the relevant user interface after search, filter, sort or cart actions.
- List filtering: The product list is filtered by matching the search text and selected category.
- List sorting: The filtered products are sorted according to the selected price or rating option.
- GridView.builder: Displays the resulting products in a scrollable grid.
- ProductCard: A reusable custom widget that displays each product's emoji, name, category, price, rating and Add to Cart button.

When the user enters text, the application compares the lowercase search query with the lowercase product name. It also checks whether each product belongs to the selected category. Only products satisfying both conditions are displayed. The sorting option then arranges the matching products by price or rating. If no products match, a "No products found!" message is displayed. The Clear Filters button resets the search text, category and sorting option. The Add to Cart button increments the cart count and displays a confirmation message.

The application was executed in Google Chrome. The screenshots show the default product list, the available sorting options and the search result when the user enters "wat", which returns Smart Watch and Wrist Watch.

ogram

File: lib/main.dart

```
import 'package:flutter/material.dart';


void main() {
  runApp(const ShopEaseApp());
}


// MAIN APPLICATION
class ShopEaseApp extends StatelessWidget {
  const ShopEaseApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'ShopEase Interactive Store',
      theme: ThemeData(
        colorSchemeSeed: Colors.deepPurple,
        useMaterial3: true,
      ),
      home: const ProductHomePage(),
    );
  }
}


// PRODUCT MODEL
class Product {
  final String name;
  final String category;
  final String emoji;
  final double price;
  final double rating;
  final Color color;


  const Product({
    required this.name,
    required this.category,
    required this.emoji,
    required this.price,
    required this.rating,
    required this.color,
  });
}


// MAIN PRODUCT PAGE
class ProductHomePage extends StatefulWidget {
  const ProductHomePage({super.key});


  @override
  State<ProductHomePage> createState() => _ProductHomePageState();
}


class _ProductHomePageState extends State<ProductHomePage> {
  // PREDEFINED PRODUCT LIST
  final List<Product> products = const [
    Product(
      name: 'Wireless Headphones',
      category: 'Electronics',
      emoji: '🎧',
      price: 1499,
      rating: 4.8,
      color: Color(0xFFEDE7F6),
    ),
    Product(
      name: 'Smart Watch',
      category: 'Electronics',
      emoji: '⌚',
```

```
    price: 2499,
    rating: 4.6,
    color: Color(0xFFE3F2FD),
  ),
  Product(
    name: 'Running Shoes',
    category: 'Fashion',
    emoji: '👟',
    price: 1999,
    rating: 4.5,
    color: Color(0xFFFFF3E0),
  ),
  Product(
    name: 'Backpack',
    category: 'Accessories',
    emoji: '🎒',
    price: 999,
    rating: 4.4,
    color: Color(0xFFE8F5E9),
  ),
  Product(
    name: 'Camera',
    category: 'Electronics',
    emoji: '📷',
    price: 5999,
    rating: 4.9,
    color: Color(0xFFFCE4EC),
  ),
  Product(
    name: 'Sunglasses',
    category: 'Fashion',
    emoji: '🕶️',
    price: 799,
    rating: 4.3,
    color: Color(0xFFFFF9C4),
  ),
  Product(
    name: 'Smartphone',
    category: 'Electronics',
    emoji: '📱',
    price: 12999,
    rating: 4.7,
    color: Color(0xFFE0F7FA),
  ),
  Product(
    name: 'Wrist Watch',
    category: 'Accessories',
    emoji: '⌚',
    price: 1799,
    rating: 4.5,
    color: Color(0xFFF3E5F5),
  ),
];


// INPUT WIDGET VARIABLES
String searchQuery = '';
String selectedCategory = 'All';
String sortOption = 'Default';
int cartCount = 0;


final TextEditingController searchController =
    TextEditingController();


// GET FILTERED PRODUCTS
List<Product> get filteredProducts {
  List<Product> result = products.where((product) {
    final matchesSearch = product.name
        .toLowerCase()
        .contains(searchQuery.toLowerCase());


    final matchesCategory =
        selectedCategory == 'All' ||
```

```
        product.category == selectedCategory;

    return matchesSearch && matchesCategory;
  }).toList();

  // SORT PRODUCTS
  if (sortOption == 'Price: Low to High') {
    result.sort((a, b) => a.price.compareTo(b.price));
  } else if (sortOption == 'Price: High to Low') {
    result.sort((a, b) => b.price.compareTo(a.price));
  } else if (sortOption == 'Rating: High to Low') {
    result.sort((a, b) => b.rating.compareTo(a.rating));
  }

  return result;
}

// CLEAR ALL FILTERS
void clearFilters() {
  setState(() {
    searchQuery = '';
    selectedCategory = 'All';
    sortOption = 'Default';
    searchController.clear();
  });
}

@override
void dispose() {
  searchController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F7FB),

    // APPLICATION BAR
    appBar: AppBar(
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
      title: const Text(
        'ShopEase',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      actions: [
        Padding(
          padding: const EdgeInsets.only(right: 20),
          child: Row(
            children: [
              const Icon(Icons.shopping_cart_outlined),
              const SizedBox(width: 5),
              Text('$cartCount'),
            ],
          ),
        ),
      ],
    ),

    // APPLICATION BODY
    body: LayoutBuilder(
      builder: (context, constraints) {
        final isWide = constraints.maxWidth >= 600;

        return Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // SEARCH INPUT
```

```
Padding(
  padding: const EdgeInsets.all(16),
  child: TextField(
    controller: searchController,
    onChanged: (value) {
      setState(() {
        searchQuery = value;
      });
    },
    decoration: InputDecoration(
      hintText: 'Search products...',
      prefixIcon: const Icon(Icons.search),
      suffixIcon: IconButton(
        icon: const Icon(Icons.clear),
        onPressed: () {
          setState(() {
            searchController.clear();
            searchQuery = '';
          });
        },
      ),
      filled: true,
      fillColor: Colors.white,
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(15),
        borderSide: BorderSide.none,
      ),
    ),
  ),
),

// FILTER SECTION
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        'Filter by Category',
        style: TextStyle(
          fontWeight: FontWeight.bold,
          fontSize: 17,
        ),
      ),
      const SizedBox(height: 10),


      // CATEGORY FILTER CHIPS
      Wrap(
        spacing: 8,
        runSpacing: 5,
        children: [
          'All',
          'Electronics',
          'Fashion',
          'Accessories',
        ].map((category) {
          return ChoiceChip(
            label: Text(category),
            selected: selectedCategory == category,
            onSelected: (selected) {
              setState(() {
                selectedCategory =
                    selected ? category : 'All';
              });
            },
          );
        }).toList(),
      ),


      const SizedBox(height: 12),


      // SORTING DROPDOWN
```

```
        Row(
          children: [
            const Text(
              'Sort by:',
              style: TextStyle(
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: DropdownButtonFormField<String>(
                initialValue: sortOption,
                decoration: InputDecoration(
                  filled: true,
                  fillColor: Colors.white,
                  contentPadding:
                      const EdgeInsets.symmetric(
                    horizontal: 12,
                    vertical: 8,
                  ),
                  border: OutlineInputBorder(
                    borderRadius:
                        BorderRadius.circular(12),
                    borderSide: BorderSide.none,
                  ),
                ),
                items: [
                  'Default',
                  'Price: Low to High',
                  'Price: High to Low',
                  'Rating: High to Low',
                ].map((option) {
                  return DropdownMenuItem(
                    value: option,
                    child: Text(
                      option,
                      style: const TextStyle(
                        fontSize: 13,
                      ),
                    ),
                  );
                }).toList(),
                onChanged: (value) {
                  setState(() {
                    sortOption = value!;
                  });
                },
              ),
            ),
          ],
        ),

        // CLEAR FILTER BUTTON
        Align(
          alignment: Alignment.centerRight,
          child: TextButton.icon(
            onPressed: clearFilters,
            icon: const Icon(Icons.refresh),
            label: const Text('Clear Filters'),
          ),
        ),
      ],
    ),
  ),

  // PRODUCT COUNT
  Padding(
    padding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 5,
    ),
    child: Text(
      '${filteredProducts.length} Products Found',
      style: const TextStyle(
```

```
                fontSize: 18,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),

          // FILTERED PRODUCT LIST
          Expanded(
            child: filteredProducts.isEmpty
                ? const Center(
                    child: Column(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [
                        Icon(
                          Icons.search_off,
                          size: 60,
                          color: Colors.grey,
                        ),
                        SizedBox(height: 10),
                        Text(
                          'No products found!',
                          style: TextStyle(fontSize: 18),
                        ),
                      ],
                    ),
                  )
                : GridView.builder(
                    padding: const EdgeInsets.all(12),
                    itemCount: filteredProducts.length,
                    gridDelegate:
                        SliverGridDelegateWithFixedCrossAxisCount(
                      crossAxisCount: isWide ? 3 : 2,
                      crossAxisSpacing: 12,
                      mainAxisSpacing: 12,
                      childAspectRatio: 0.68,
                    ),
                    itemBuilder: (context, index) {
                      final product = filteredProducts[index];

                      return ProductCard(
                        product: product,
                        onAddToCart: () {
                          setState(() {
                            cartCount++;
                          });

                          ScaffoldMessenger.of(context)
                              .showSnackBar(
                            SnackBar(
                              content: Text(
                                '${product.name} added to cart!',
                              ),
                              duration:
                                  const Duration(seconds: 1),
                            ),
                          );
                        },
                      );
                    },
                  ),
          ),
        ],
      );
    },
  ),
 );
 }
}

// REUSABLE PRODUCT CARD
class ProductCard extends StatelessWidget {
  final Product product;
```

```
final VoidCallback onAddToCart;

const ProductCard({
  super.key,
  required this.product,
  required this.onAddToCart,
});

@override
Widget build(BuildContext context) {
  return Card(
    elevation: 3,
    color: Colors.white,
    clipBehavior: Clip.antiAlias,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(15),
    ),
    child: Padding(
      padding: const EdgeInsets.all(10),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // PRODUCT IMAGE
          Expanded(
            child: Container(
              width: double.infinity,
              decoration: BoxDecoration(
                color: product.color,
                borderRadius: BorderRadius.circular(12),
              ),
              child: Center(
                child: Text(
                  product.emoji,
                  style: const TextStyle(fontSize: 50),
                ),
              ),
            ),
          ),

          const SizedBox(height: 8),

          // PRODUCT NAME
          Text(
            product.name,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: const TextStyle(
              fontSize: 14,
              fontWeight: FontWeight.bold,
            ),
          ),

          const SizedBox(height: 4),

          // CATEGORY
          Text(
            product.category,
            style: const TextStyle(
              color: Colors.grey,
              fontSize: 12,
            ),
          ),

          const SizedBox(height: 5),

          // PRICE AND RATING
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text(
```

```
                '₹${product.price.toStringAsFixed(0)}',
                style: const TextStyle(
                  color: Colors.deepPurple,
                  fontWeight: FontWeight.bold,
                ),
              ),
              Row(
                children: [
                  const Icon(
                    Icons.star,
                    color: Colors.amber,
                    size: 15,
                  ),
                  Text(
                    product.rating.toString(),
                    style: const TextStyle(fontSize: 12),
                  ),
                ],
              ),
            ],
          ),

          const SizedBox(height: 8),

          // ADD TO CART BUTTON
          SizedBox(
            width: double.infinity,
            height: 35,
            child: ElevatedButton(
              onPressed: onAddToCart,
              style: ElevatedButton.styleFrom(
                backgroundColor: Colors.deepPurple,
                foregroundColor: Colors.white,
                padding: EdgeInsets.zero,
              ),
              child: const Text(
                'Add to Cart',
                style: TextStyle(fontSize: 12),
              ),
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

Output Screenshots

The following screenshots show the ShopEase Interactive Store running in Google Chrome. Figure 1 shows the default product list and the available category filters. Figure 2 shows the sorting dropdown with the available sorting choices. Figure 3 shows the search operation for the text “wat”, which displays two matching products: Smart Watch and Wrist Watch.

Figure 1: Default product list with category filters

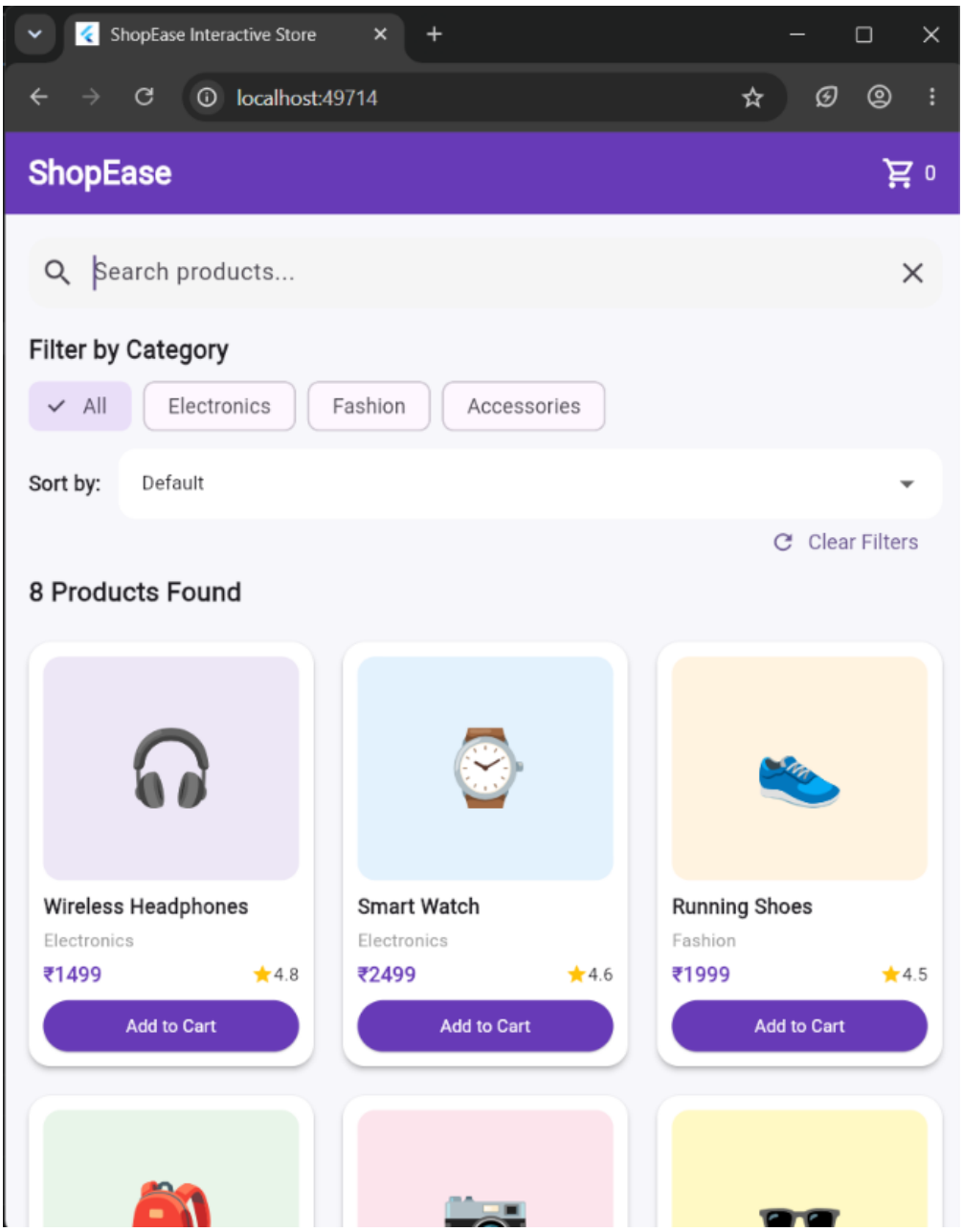
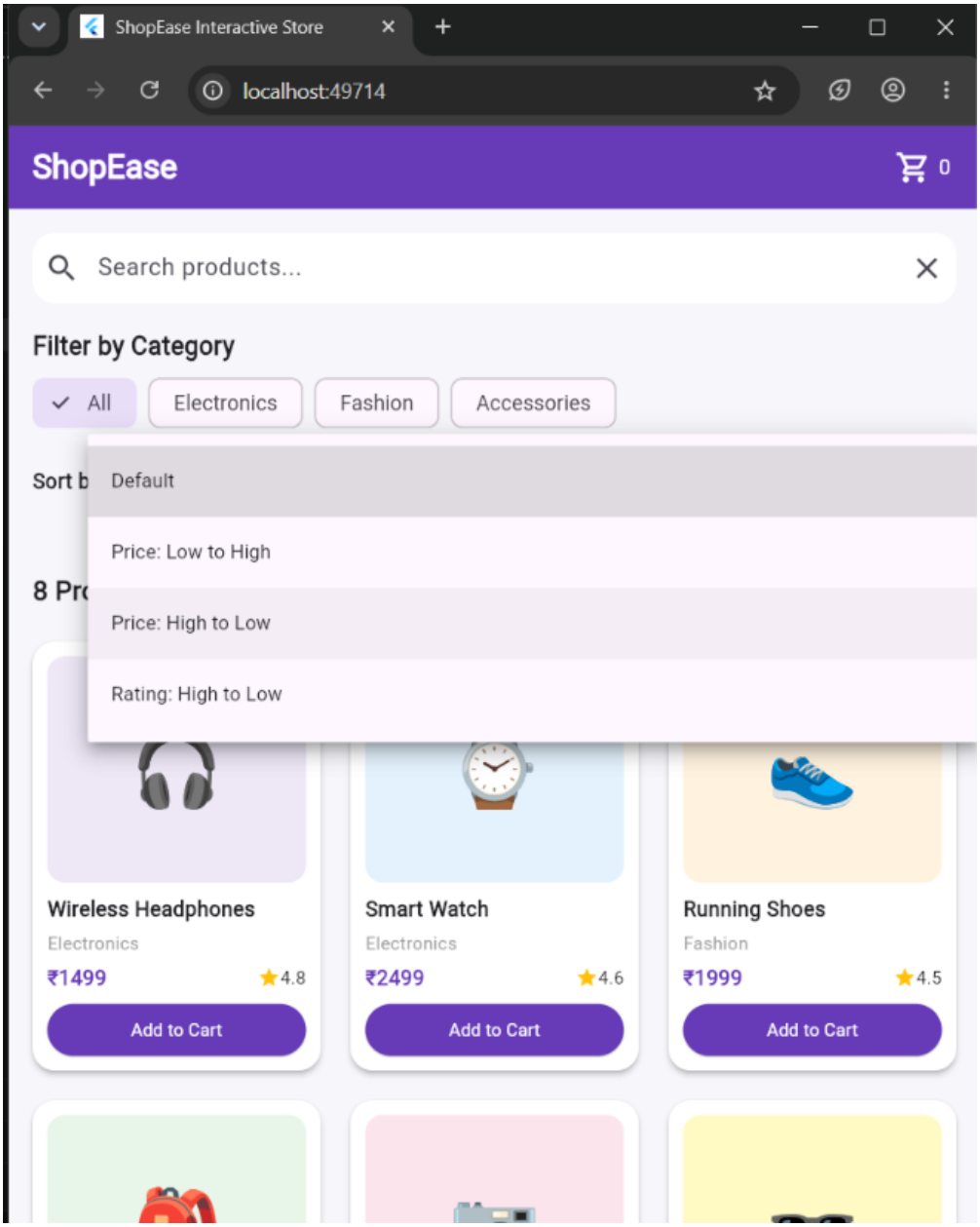
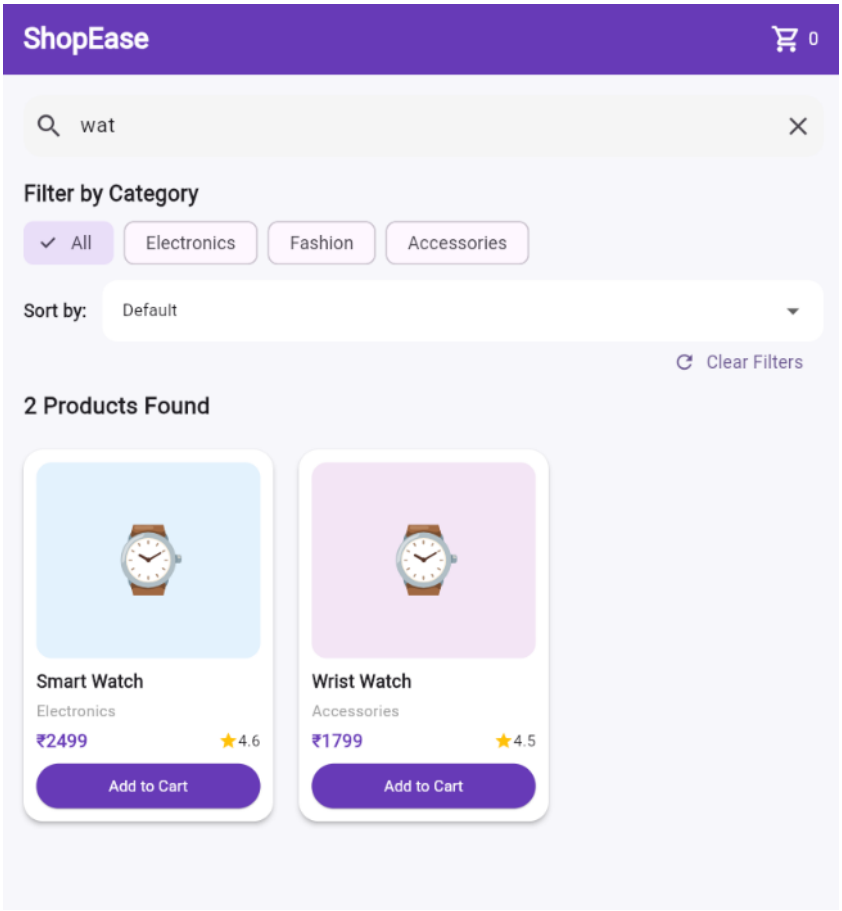


Figure 2: Sorting options displayed in the dropdown


ShopEase Interactive Store
localhost:49714
ShopEase
0
Search products...
Filter by Category
All
Electronics
Fashion
Accessories
Sort b
Default
Price: Low to High
Price: High to Low
Rating: High to Low
8 Pr
Wireless Headphones
Electronics
₹1499
4.8
Add to Cart
Smart Watch
Electronics
₹2499
4.6
Add to Cart
Running Shoes
Fashion
₹1999
4.5
Add to Cart

Output Screenshots (Continued)

Figure 3: Search result for the query "wat"



Result

Thus, the interactive e-commerce product catalog application "ShopEase Interactive Store" was successfully developed using Flutter. The application displays a predefined product list and allows users to search products, filter them by category and sort them by price or rating using input widgets. The Clear Filters option resets the selections, and the Add to Cart action updates the cart count. The application was executed successfully in Google Chrome.